

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital VLSI Testing (01CT0629)	Aim: Aim: To implement a 2-bit Synchronous Counter using D Flip-Flops and observe SA0 and SA1 Faults.	
Experiment No: 9	Date: 03-04-2026	Enrolment No: 924301733056

AIM: To implement a 2-bit Synchronous Counter using D Flip-Flops and observe SA0 and SA1 Faults.

IDE: EDA Playground

THEORY:

A synchronous counter is a sequential digital circuit in which all flip-flops are triggered simultaneously by a common clock signal. In this experiment, a 2-bit synchronous counter is implemented using D flip-flops. The counter produces a binary sequence of states (00, 01, 10, 11) on every clock pulse. The design is based on the characteristic equation of the D flip-flop, where the next state is directly given by the input D. For a 2-bit counter, the least significant bit (Q0) toggles on every clock cycle, so its input is given by $D0 = \overline{Q0}$, while the most significant bit (Q1) toggles only when Q0 is high, hence $D1 = Q1 \oplus Q0$. Since all flip-flops are clocked together, the counter operates faster and avoids propagation delays seen in asynchronous counters. Additionally, fault analysis is performed using stuck-at faults such as SA0 (stuck-at-0) and SA1 (stuck-at-1), where a signal line is permanently fixed at logic 0 or 1. These faults disturb the normal counting sequence, making it possible to detect errors in the circuit behavior during simulation.

DESIGN AND IMPLEMENTATION:

$$D0 = \sim Q0$$

$$D1 = Q1 \wedge Q0$$

VERILOG CODE:

A. Verilog Code:

```
// Code your design here
module sync_counter_fault (
    input clk,
    input reset,
    input SA0_Q0, // Force Q0 = 0
    input SA1_Q0, // Force Q0 = 1
    input SA0_Q1, // Force Q1 = 0
    input SA1_Q1, // Force Q1 = 1
    output reg Q1,
    output reg Q0
);

reg D0, D1;

always @(*) begin
    D0 = ~Q0;
    D1 = Q1 ^ Q0;
end
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital VLSI Testing (01CT0629)	Aim: Aim: To implement a 2-bit Synchronous Counter using D Flip-Flops and observe SA0 and SA1 Faults.	
Experiment No: 9	Date: 03-04-2026	Enrolment No: 924301733056

```

always @(posedge clk or posedge reset)
begin
    if (reset) begin
        Q1 <= 0;
        Q0 <= 0;
    end
    else begin
        Q0 <= (SA0_Q0) ? 0 :
            (SA1_Q0) ? 1 : D0;

        Q1 <= (SA0_Q1) ? 0 :
            (SA1_Q1) ? 1 : D1;
    end
end

endmodule

```

B. Testbench:

```

// Code your testbench here
// or browse Examples
module tb_sync_counter;

    reg clk, reset;
    reg SA0_Q0, SA1_Q0, SA0_Q1, SA1_Q1;
    wire Q1, Q0;

    // Instantiate module
    sync_counter_fault uut (
        .clk(clk),
        .reset(reset),
        .SA0_Q0(SA0_Q0),
        .SA1_Q0(SA1_Q0),
        .SA0_Q1(SA0_Q1),
        .SA1_Q1(SA1_Q1),
        .Q1(Q1),
        .Q0(Q0)
    );

    // Clock generation
    always #5 clk = ~clk;

    initial begin
        // Dump file for waveform
        $dumpfile("counter_waveform.vcd"); // file name
        $dumpvars(0, tb_sync_counter);    // dump all variables

        $monitor("Time=%0t | Q1=%b Q0=%b", $time, Q1, Q0);

        clk = 0;
        reset = 1;
        SA0_Q0 = 0; SA1_Q0 = 0;
        SA0_Q1 = 0; SA1_Q1 = 0;
    end
endmodule

```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital VLSI Testing (01CT0629)	Aim: Aim: To implement a 2-bit Synchronous Counter using D Flip-Flops and observe SA0 and SA1 Faults.	
Experiment No: 9	Date: 03-04-2026	Enrolment No: 924301733056

```

#10 reset = 0;

// Normal operation
#40;

// Apply SA0 fault on Q0
SA0_Q0 = 1;
#20;
SA0_Q0 = 0;

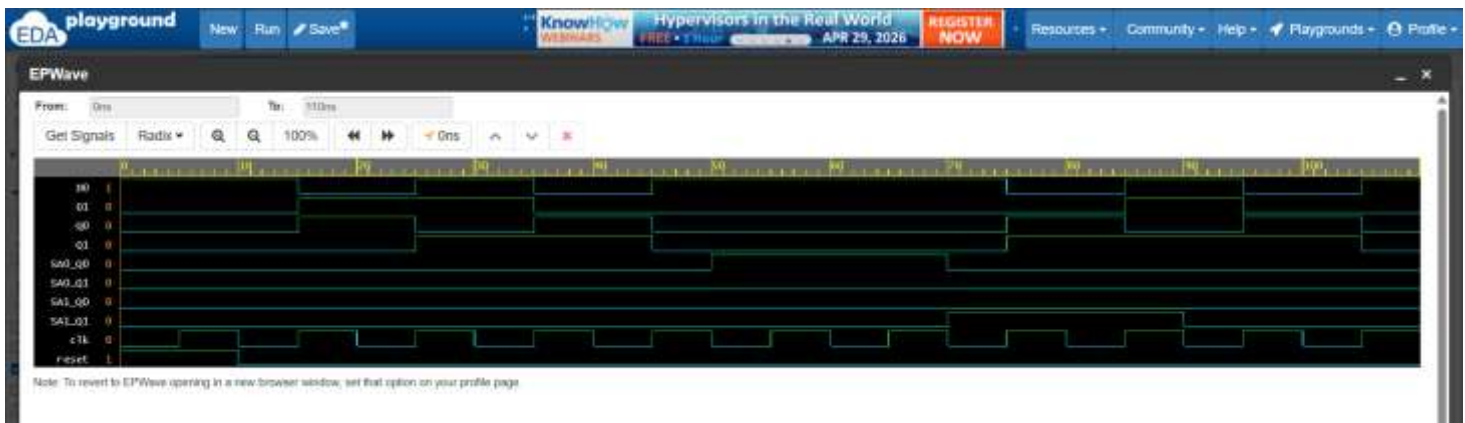
// Apply SA1 fault on Q1
SA1_Q1 = 1;
#20;
SA1_Q1 = 0;

#20 $finish;
end

endmodule

```

SIMULATION RESULTS:



CONCLUSION:

The 2-bit synchronous counter using D flip-flops was successfully designed and implemented. The counter produced the expected binary sequence (00 → 01 → 10 → 11) with all flip-flops operating simultaneously under a common clock signal, ensuring faster and reliable performance. Fault analysis using stuck-at-0 (SA0) and stuck-at-1 (SA1) conditions was also carried out, and it was observed that these faults disrupted the normal counting sequence. This helped in understanding how faults affect digital circuits and how they can be detected through simulation. Hence, the experiment verified both the correct operation of the synchronous counter and the impact of faults in digital systems.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital VLSI Testing (01CT0629)	Aim: Aim: To implement a 2-bit Synchronous Counter using D Flip-Flops and observe SA0 and SA1 Faults.	
Experiment No: 9	Date: 03-04-2026	Enrolment No: 924301733056

Post Lab Exercise:

To design a 3-bit Synchronous counter using D flipflop.

Design code:

```

module sync_counter_3bit (
    input clk,
    input reset,
    output reg Q2, Q1, Q0
);

always @(posedge clk or posedge reset)
begin
    if (reset) begin
        Q2 <= 0;
        Q1 <= 0;
        Q0 <= 0;
    end
    else begin
        Q0 <= ~Q0;
        Q1 <= Q1 ^ Q0;
        Q2 <= Q2 ^ (Q1 & Q0);
    end
end

endmodule

```

Testbench code:

```

module tb_sync_counter_3bit;

reg clk, reset;
wire Q2, Q1, Q0;

// Instantiate DUT
sync_counter_3bit uut (
    .clk(clk),
    .reset(reset),
    .Q2(Q2),
    .Q1(Q1),
    .Q0(Q0)
);

// Clock generation
always #5 clk = ~clk;

```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital VLSI Testing (01CT0629)	Aim: Aim: To implement a 2-bit Synchronous Counter using D Flip-Flops and observe SA0 and SA1 Faults.	
Experiment No: 9	Date: 03-04-2026	Enrolment No: 924301733056

```

initial begin
    // Waveform dump
    $dumpfile("counter3bit.vcd");
    $dumpvars(0, tb_sync_counter_3bit);

    $monitor("Time=%0t | Q2=%b Q1=%b Q0=%b", $time, Q2, Q1, Q0);

    clk = 0;
    reset = 1;

    #10 reset = 0;

    // Run for some cycles
    #100;

    $finish;
end

endmodule

```

Output:

